L12 — Recap and Comparative Analysis of Sorting Techniques

Unit: 1 | **Chapter:** Sorting Techniques | **BT Level:** BT4 | **CO:** CO1, CO5

Learning Objectives

- Compare all sorting algorithms covered on time, space, stability, and adaptability
- Select the appropriate sorting algorithm for a given scenario
- Understand the theoretical lower bound for comparison-based sorting
- Analyze real-world sorting library implementations

---

1. Why Compare Sorting Algorithms?

Different sorting algorithms excel in different contexts:

- **Small arrays:** Insertion sort often beats merge sort despite $O(n^2)$ theory
- **Nearly sorted data:** Insertion sort shines (O(n))
- **Memory-constrained systems:** In-place algorithms (quick sort, heap sort) preferred
- **Stability required:** Merge sort or Timsort

---

2. Comprehensive Comparison Table

Algorithm	Best	Average	Worst	Space	Stable	Adaptive	Notes
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	✓	✓	Simple but slow
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	✗	✗	Min swaps
Insertion Sort	**O(n)**	$O(n^2)$	$O(n^2)$	O(1)	✓	✓	Best for small/near-sorted
Merge Sort	O(n log n)	O(n log n)	**O(n log n)**	**O(n)**	✓	✗	Guaranteed; external sort
Quick Sort	O(n log n)	**O(n log n)**	$O(n^2)$	O(log n)	✗	✗	Fastest in practice
Heap Sort	O(n log n)	O(n log n)	O(n log n)	**O(1)**	✗	✗	In-place O(n log n)
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	✓	—	Integer keys only
Radix Sort	O(d·n)	O(d·n)	O(d·n)	O(n + k)	✓	—	Fixed-digit keys
Bucket Sort	O(n + k)	O(n + k)	$O(n^2)$	O(n)	✓	—	Uniform distribution

---

3. Theoretical Lower Bound for Comparison Sort

Theorem: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof sketch using decision tree:

- A decision tree has one leaf for each possible permutation of n elements → n! leaves
- A binary tree with L leaves has height $\geq \log_2(L)$
- Therefore, worst-case comparisons $\geq \log_2(n!) = \Theta(n \log n)$ by Stirling's approximation

$$\log_2(n!) = \sum_{k=1}^{n}\log_2(k) \approx n\log_2 n - n\log_2 e = \Theta(n \log n)$$

Implication: Merge sort, quick sort, and heap sort are asymptotically **optimal** comparison sorts.

Non-comparison sorts (Counting, Radix, Bucket) break this barrier by exploiting numeric properties.

---

4. Algorithm Selection Guide

4.1 By Input Characteristics

Input	Best Choice
Nearly sorted (few inversions)	Insertion Sort
Uniformly distributed floats [0,1)	Bucket Sort
Integers in small range [0, k]	Counting Sort
Integers with fixed digits	Radix Sort
General (unknown distribution)	Quick Sort (random pivot)
Guaranteed O(n log n) needed	Merge Sort

4.2 By Constraint

Constraint	Best Choice
O(1) space required	Heap Sort
Stability required	Merge Sort
Linked list input	Merge Sort
Small n (< 50)	Insertion Sort
External sorting (data on disk)	Merge Sort
Parallel sorting	Merge Sort

---

5. Counting Inversions (Sorting Quality Metric)

An **inversion** is a pair (i, j) where $i < j$ but $arr[i] > arr[j]$.

- 0 inversions → already sorted (best case for insertion sort)
- n(n-1)/2 inversions → reverse sorted (worst case)

Count inversions using merge sort in O(n log n):

```
def merge_count(arr, temp, left, mid, right):
    i, j, k = left, mid + 1, left
    inv_count = 0
    while i <= mid and j <= right:
        if arr[i] <= arr[j]:
            temp[k] = arr[i]
            i += 1
        else:
            temp[k] = arr[j]
            inv_count += (mid - i + 1)  # All remaining left elements f
            j += 1
        k += 1
    while i <= mid:
        temp[k] = arr[i]; i += 1; k += 1
    while j <= right:
        temp[k] = arr[j]; j += 1; k += 1
    for i in range(left, right + 1):
        arr[i] = temp[i]
    return inv_count
```

---

6. Python's Built-in Sort: Timsort

Python's `list.sort()` and `sorted()` use **Timsort** (Tim Peters, 2002):

- Hybrid of **merge sort** and **insertion sort**
- Identifies naturally sorted "runs" in the input
- Merges runs using merge sort

Property	Timsort
Best	O(n)
Average	O(n log n)
Worst	O(n log n)
Space	O(n)
Stable	Yes

```
arr = [5, 2, 9, 1, 5, 6]
arr.sort()          # In-place, Timsort
arr = sorted(arr)   # Returns new list, Timsort
```

---

7. Practical Performance on Real Data

For n = 100,000 random integers (approximate):

Algorithm	Time (ms)
Insertion Sort	~5,000
Merge Sort	~30

Quick Sort (random pivot)	~20
Heap Sort	~50
Python sorted()	~10

Note: Real performance depends heavily on constants, cache behavior, and language.

---

8. Summary Cheat Sheet

```
Need stable sort?
  ├─ Yes → Merge Sort (or Timsort for Python)
  └─ No  → Quick Sort (general), Heap Sort (in-place O(n log n))

Have integer keys?
  ├─ Small range [0, k] → Counting Sort O(n+k)
  ├─ Fixed digits → Radix Sort
  └─ Floating-point, uniform → Bucket Sort

Input characteristics?
  ├─ Nearly sorted → Insertion Sort
  ├─ Small (n < 50) → Insertion Sort
  └─ Large, general → Quick Sort / Merge Sort

Memory constraints?
  ├─ O(1) space → Heap Sort or Quick Sort (in-place)
  └─ O(n) space OK → Merge Sort
```

---

9. Key Takeaways

1. No single sorting algorithm is best for all cases.
2. Comparison-based sorts cannot beat Ω(n log n) in worst case.
3. Non-comparison sorts break this barrier for specific input types.
4. In practice: quick sort for general use, merge sort for stability/external sort, insertion sort for small or nearly-sorted arrays.
5. Modern languages use Timsort (Python), IntroSort (C++, Java) — hybrid algorithms.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 8 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 3 (Springer, 2024)